

1. Gaussian Function (Manual Gaussian Filter)

```
import cv2
```

```
import numpy as np
```

```
img = cv2.imread(r"C:/PROGRAM/Open CV/zara_002_2026-02-26 at 9.40.10 AM.jpeg",  
0)
```

```
k = np.array([[1,2,1],  
              [2,4,2],  
              [1,2,1]], dtype=float)
```

```
k = k / 16
```

```
r, c = img.shape
```

```
out = np.zeros((r, c), dtype=np.uint8)
```

```
for i in range(1, r-1):
```

```
    for j in range(1, c-1):
```

```
        s = 0
```

```
        for x in range(3):
```

```
            for y in range(3):
```

```
                s += img[i+x-1, j+y-1] * k[x, y]
```

```
        out[i, j] = int(s)
```

```
cv2.imshow("Original", img)
```

```
cv2.imshow("Gaussian", out)
```

```
cv2.waitKey(0)
```

```
cv2.destroyAllWindows()
```

Original



Output(Gaussian):



2. Gray Level Transformation (Negative)

```
import cv2
```

```
import numpy as np
```

```
img = cv2.imread(r"C:\PROGRAM\Open CV\zaraberry04.jpeg", 0)
```

```
r, c = img.shape
```

```
out = np.zeros((r, c), dtype=np.uint8)
```

```
for i in range(r):  
    for j in range(c):  
        out[i, j] = 255 - img[i, j]
```

```
cv2.imshow("Original", img)  
cv2.imshow("Negative", out)  
cv2.waitKey(0)  
cv2.destroyAllWindows()
```

Original



Output(negative)



3. Log Transformation

```
import cv2
```

```
import numpy as np

img = cv2.imread(r"C:\PROGRAM\Open CV\zaraberry00.jpeg", 0)

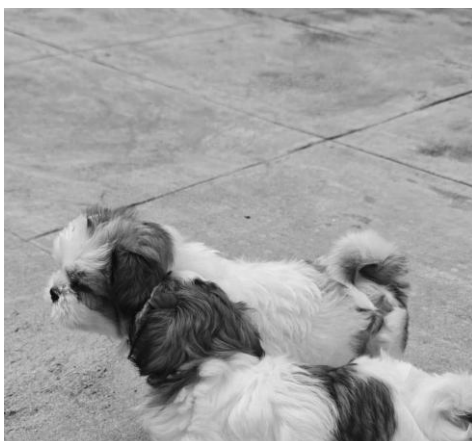
c = 255 / np.log(256)

r, col = img.shape
out = np.zeros((r, col), dtype=np.uint8)

for i in range(r):
    for j in range(col):
        out[i, j] = c * np.log(1 + img[i, j])

cv2.imshow("Original", img)
cv2.imshow("Log", out)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

Original



Output(log)



4. Gamma (Power Law) Transformation

```
import cv2
```

```
import numpy as np
```

```
img = cv2.imread("C:\PROGRAM\Open CV\zaraberry02.jpeg", 0)
```

```
g = 0.5
```

```
r, c = img.shape
```

```
out = np.zeros((r, c), dtype=np.uint8)
```

```
for i in range(r):
```

```
    for j in range(c):
```

```
        out[i, j] = np.clip(255 * ((img[i, j] / 255) ** g), 0, 255)
```

```
cv2.imshow("Original", img)
```

```
cv2.imshow("Gamma", out)
```

```
cv2.waitKey(0)
```

```
cv2.destroyAllWindows()
```

original



output(gamma)



5. Histogram Processing (Manual Histogram Equalization)

```
import cv2
```

```
import numpy as np
```

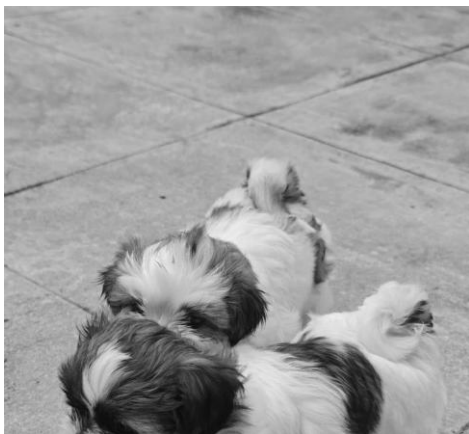
```
img = cv2.imread("C:\PROGRAM\Open CV\zaraberry03.jpeg", 0)
```

```
hist = np.zeros(256)
```

```
for p in img.flatten():
```

```
    hist[p] += 1
```

```
pdf = hist / img.size  
cdf = np.cumsum(pdf)  
  
table = np.round(cdf * 255).astype(np.uint8)  
  
out = table[img]  
  
cv2.imshow("Original", img)  
cv2.imshow("Equalized", out)  
cv2.waitKey(0)  
cv2.destroyAllWindows()  
Original
```



Equalized



6. Mean Filter (Manual Spatial Filtering)

```
import cv2
```

```
import numpy as np
```

```
img = cv2.imread("C:\PROGRAM\Open CV\zaraberry04.jpeg", 0)
```

```
r, c = img.shape
```

```
out = np.zeros((r, c), dtype=np.uint8)
```

```
for i in range(1, r-1):
```

```
    for j in range(1, c-1):
```

```
        s = 0
```

```
        for x in range(-1, 2):
```

```
            for y in range(-1, 2):
```

```
                s += int(img[i+x, j+y])
```

```
        out[i, j] = s // 9
```

```
cv2.imshow("Original", img)
```

```
cv2.imshow("Mean Filter", out)
```

```
cv2.waitKey(0)
```

```
cv2.destroyAllWindows()
```

original



Output (mean filter)



7. Laplacian Filter (Manual)

```
import cv2
```

```
import numpy as np
```

```
img = cv2.imread("C:\PROGRAM\Open CV\zaraberry00.jpeg", 0)
```

```
img = img.astype(np.int32)
```

```
k = [[0,-1,0],
```

```
      [-1,4,-1],
```

```
      [0,-1,0]]
```

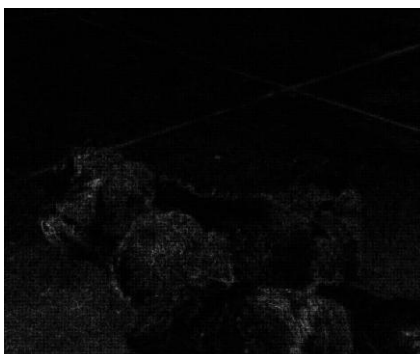
```
r, c = img.shape
out = np.zeros((r, c), dtype=np.uint8)
for i in range(1, r-1):
    for j in range(1, c-1):
        s = 0
        for x in range(3):
            for y in range(3):
                s += img[i+x-1, j+y-1] * k[x][y]

        out[i, j] = np.clip(abs(s), 0, 255)
cv2.imshow("Original", img.astype(np.uint8))
cv2.imshow("Laplacian", out)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

Original



Laplacian



8. Sobel Filter (Manual)

```
import cv2

import numpy as np

img = cv2.imread("C:\PROGRAM\Open CV\zaraberry02.jpeg", 0)
img = img.astype(np.int32)

gx = [[-1,0,1],
      [-2,0,2],
      [-1,0,1]]

gy = [[-1,-2,-1],
      [0,0,0],
      [1,2,1]]

r, c = img.shape
out = np.zeros((r, c), dtype=np.uint8)

for i in range(1, r-1):
    for j in range(1, c-1):
        sx = 0
        sy = 0

        for x in range(3):
            for y in range(3):
                p = img[i+x-1, j+y-1]
                sx += p * gx[x][y]
```

```
sy += p * gy[x][y]
```

```
g = int(np.sqrt(sx*sx + sy*sy))
```

```
out[i, j] = np.clip(g, 0, 255)
```

```
cv2.imshow("Original", img.astype(np.uint8))
```

```
cv2.imshow("Sobel", out)
```

```
cv2.waitKey(0)
```

```
cv2.destroyAllWindows()
```

Original



Output(sobel)



